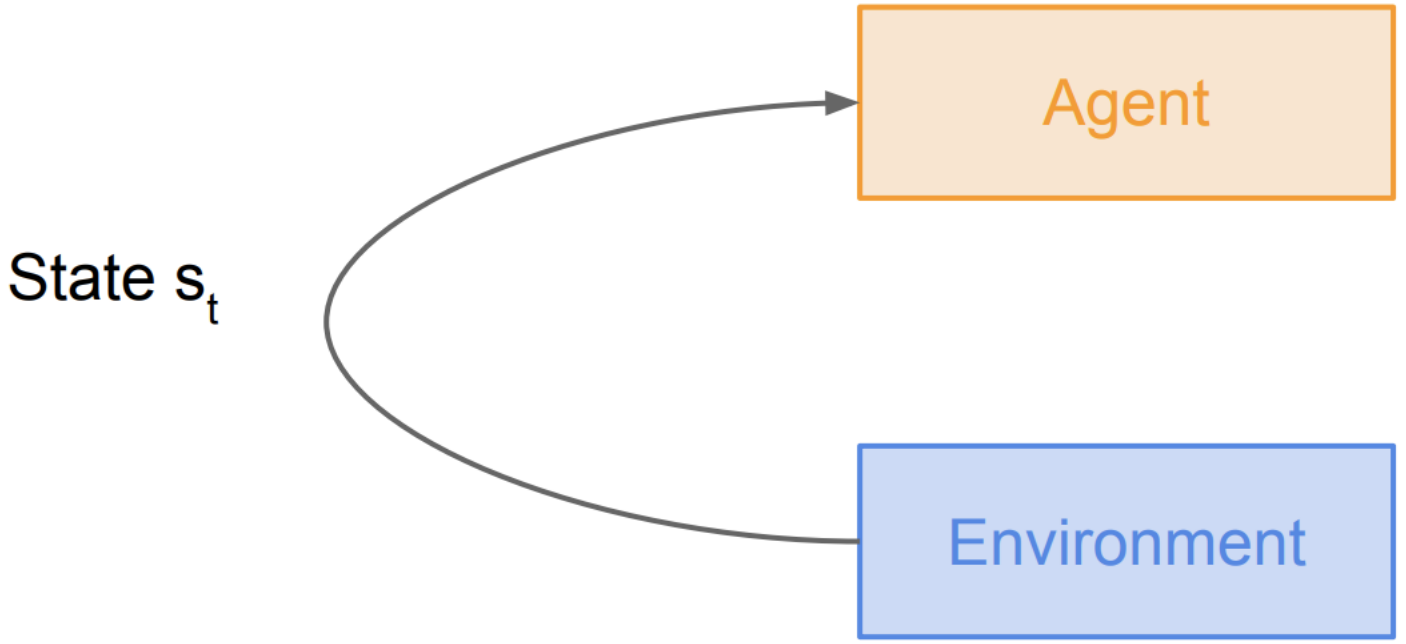
Invatare prin recompensa (Reinforcement Learning)

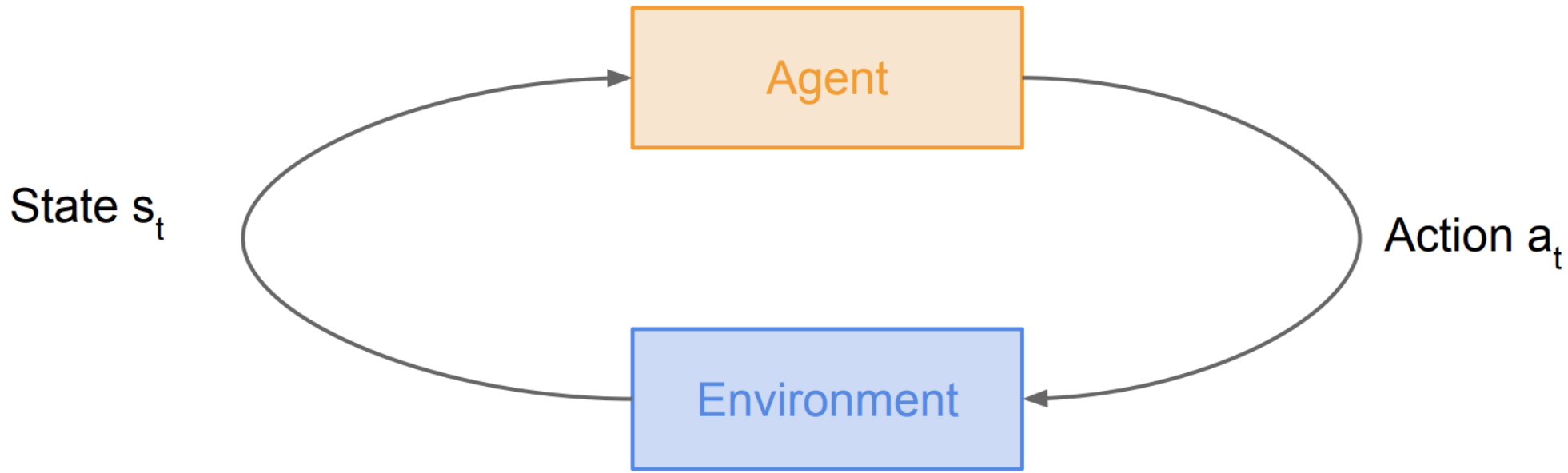
- **Invatare prin recompensa:**
  - Un agent actioneaza in mediu
  - Primeste o recompensa (evaluarea actiunii sale, fara sa i se indice care actiune este corecta pentru atingerea scopului)
  - Actiunile sunt invatate prin interactiunea cu mediului in vederea obtinerii scopului dorit

Invatare prin recompensa (RL)

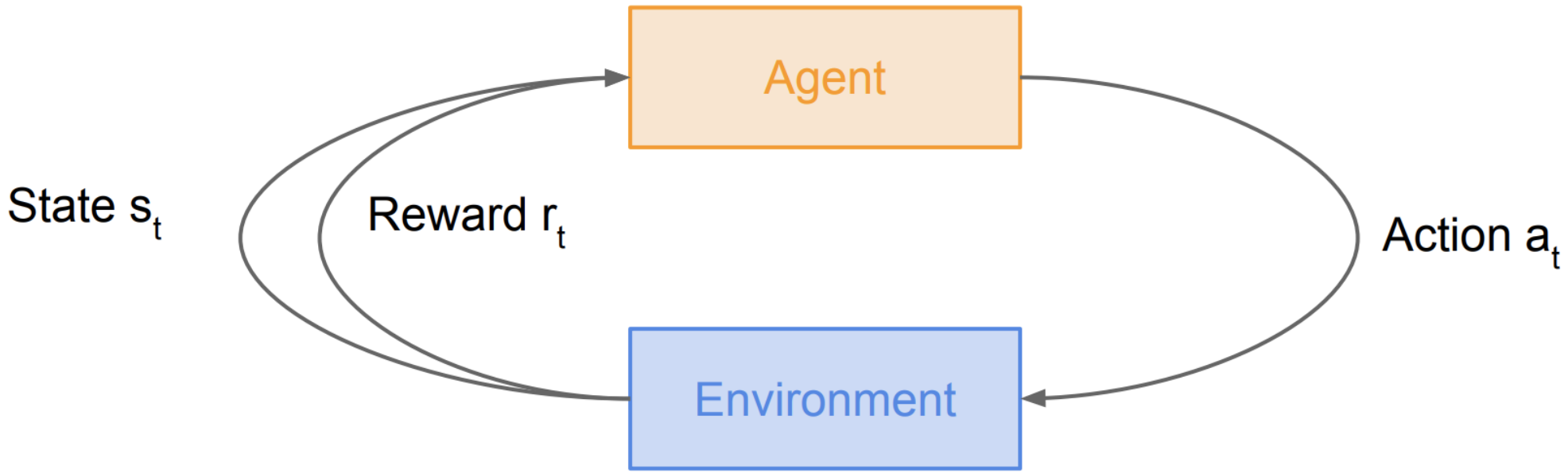
Invatare prin recompensa



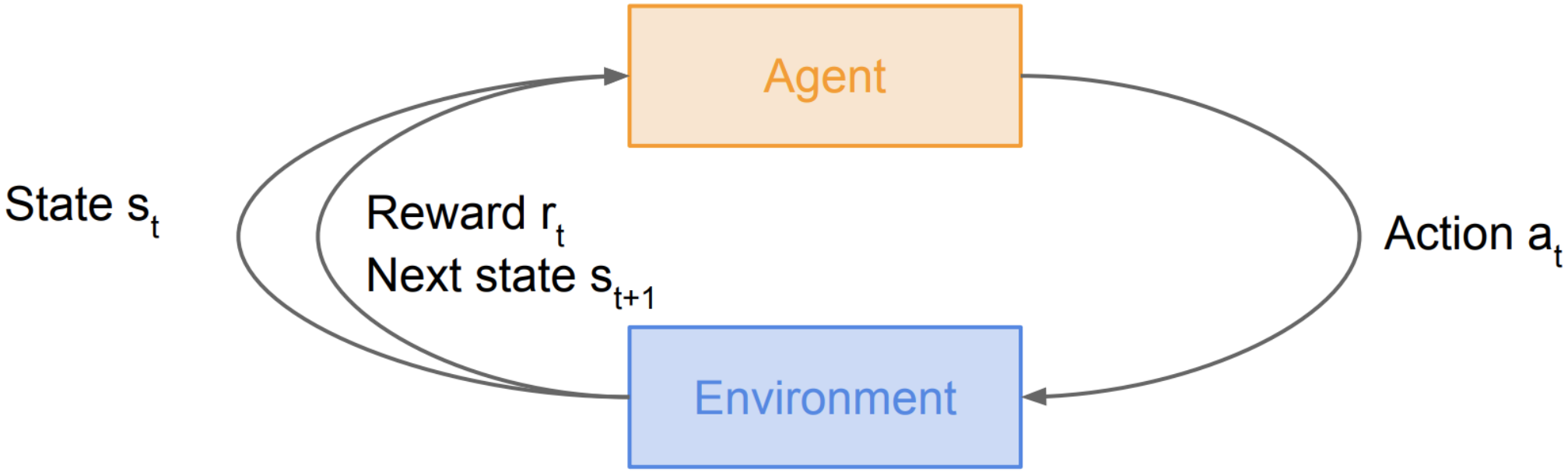
Invatare prin recompensa



Invatare prin recompensa

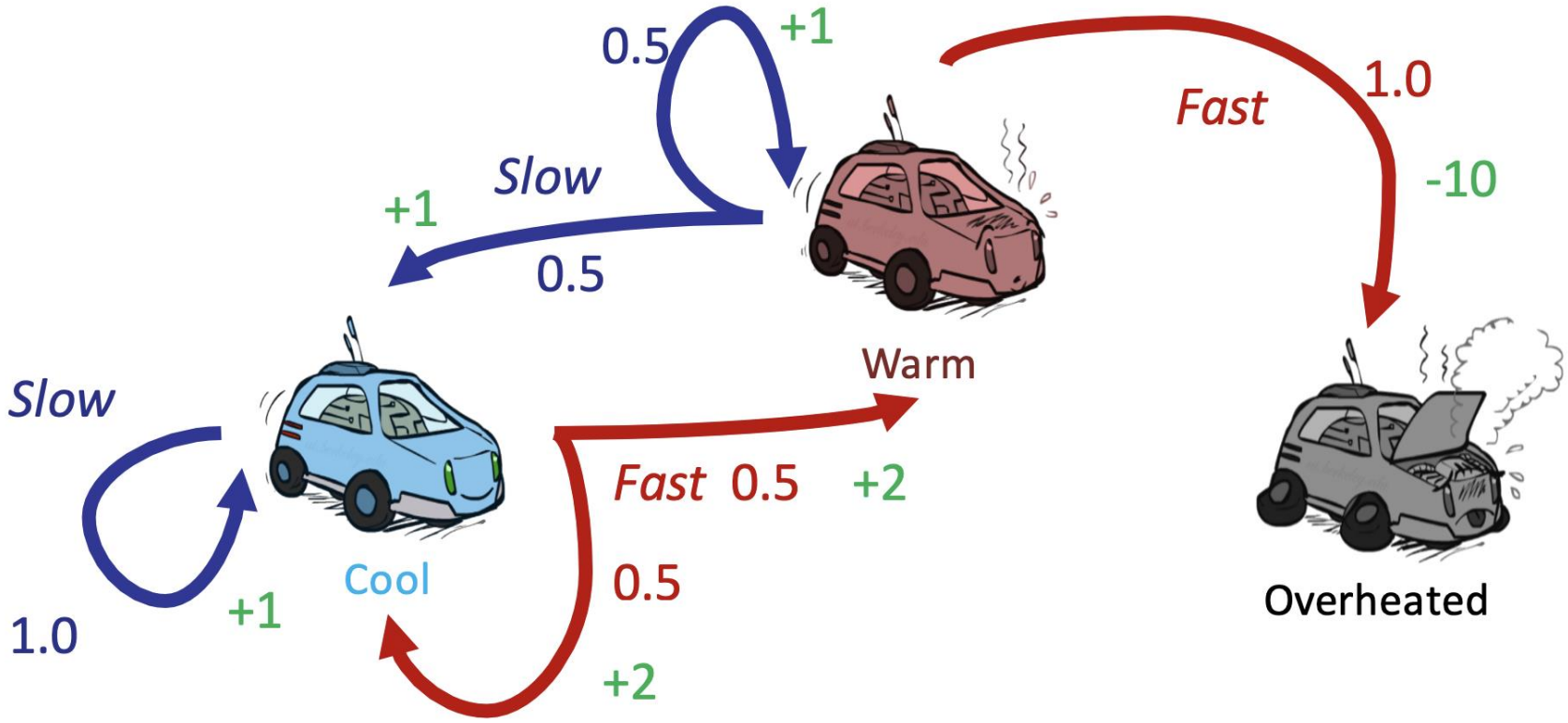


Invatare prin recompensa



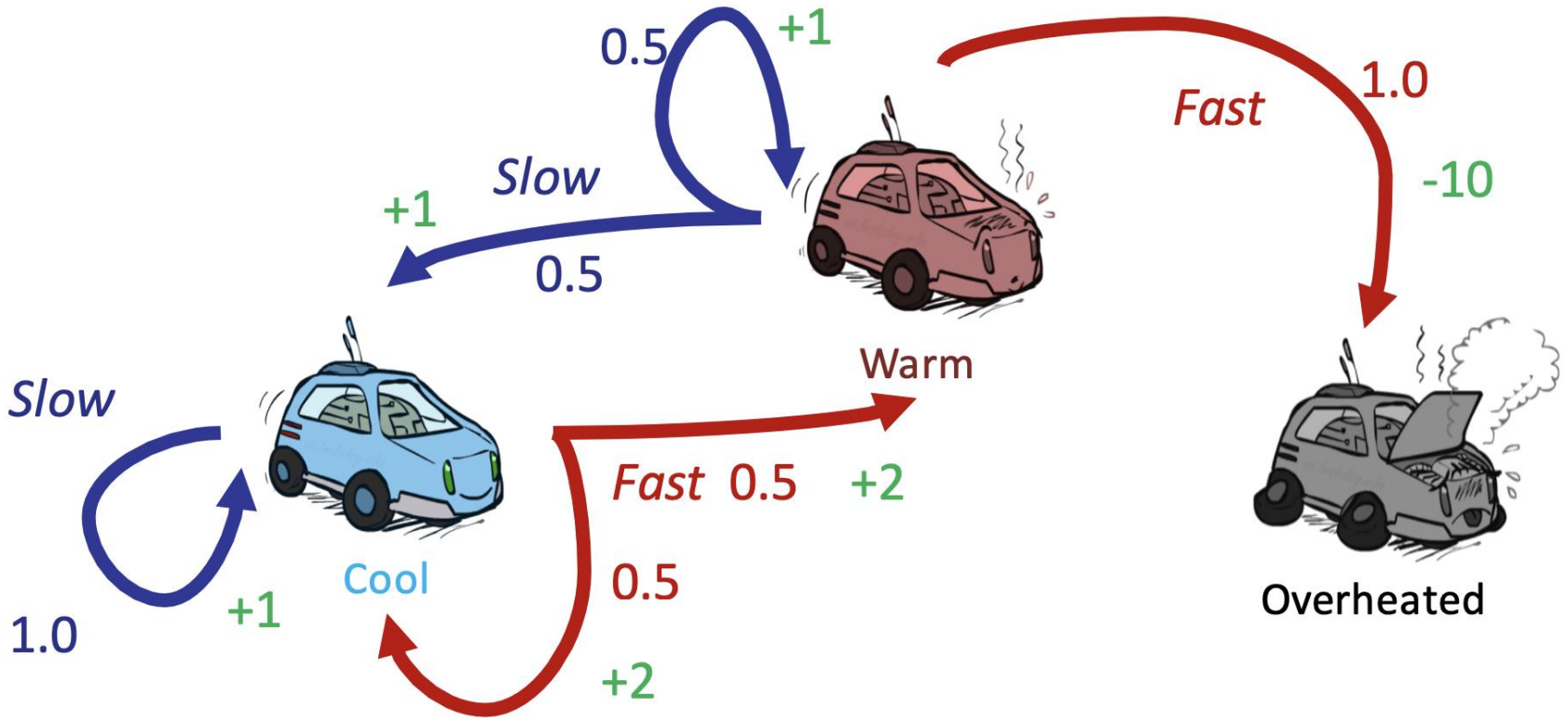
Invatare prin recompensa

Markov Decision Process



Markov Decision Process

- Stari: Cool, Warm, Overheated
- Actiuni: Slow, Fast
- Probabilitati
- Recompense



Invatare prin recompensa

Markov Decision Process

- Formulare matematica a problemei de invatare prin recompensa
- Proprietatea Markov : starea curenta caracterizeaza starea lumii

- Definit de: (S, A, R, P, gama)

- S: multimea starilor
- A: multimea actiunilor posibile
- R: recompensa (pentru perechea stare, actiune)
- P: probabilitatea tranzitiilor
- γ sau δ : factor de discount

Invatare prin recompensa

Markov Decision Process

- La momentul t=0, se alege starea s_0
- Se repeta pana la finalizare
  - Agentul selecteaza actiunea a_t
  - Mediul furnizeaza recompensa r_t
  - Agentul primeste recompensa r_t si trece in starea urmatoare s_{t+1}

- O politica **π** este o functie **π** : S -> A care specifica actiunea care va fi luata in fiecare stare

- Obiectiv: determinarea politicii optime **π*** astfel incat sa se maximizeze recompensa cumulata

Invatare prin recompensa

- Se porneste din starea s_0, se aplica **politica π**, se obtine o secventa de stari s_0, s_1, … s_t si recompense r_0, r_1, … r_t

- Utilitatea - combinare a recompenselor imediate
- **Utilitatea -** utilitatea asteptata de politica π pornind din starea s_0 trecand prin starile obtinute prin aplicarea politicii π

$$\sum_{\forall \text{ state sequences starting from } s_0} P^{\pi}(\text{sequence}) U(\text{sequence})$$

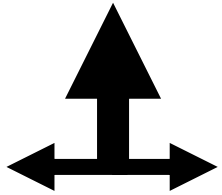
Deplasare robot intr-un grid

			+1
			-1
START			

actiuni: UP, DOWN, LEFT, RIGHT

UP

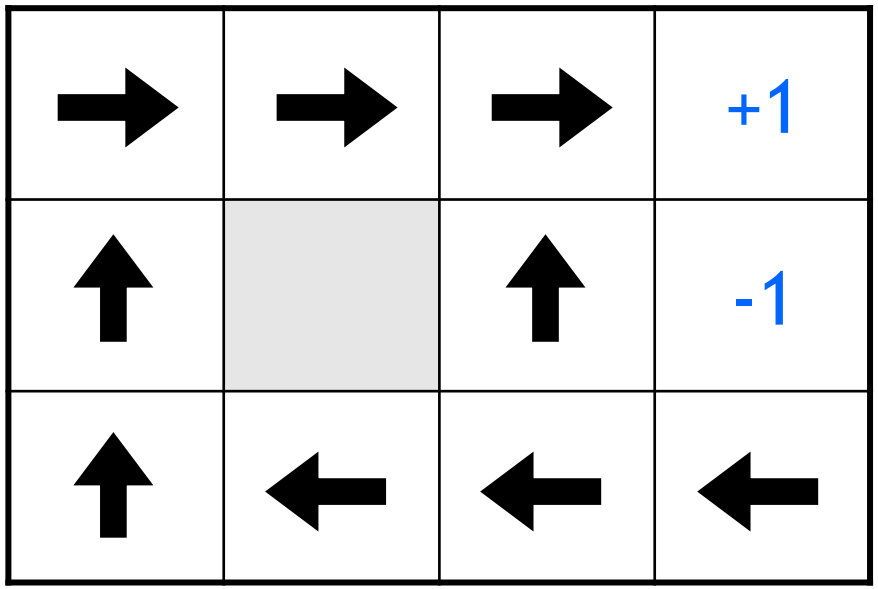
80% UP
10% LEFT
10% RIGHT

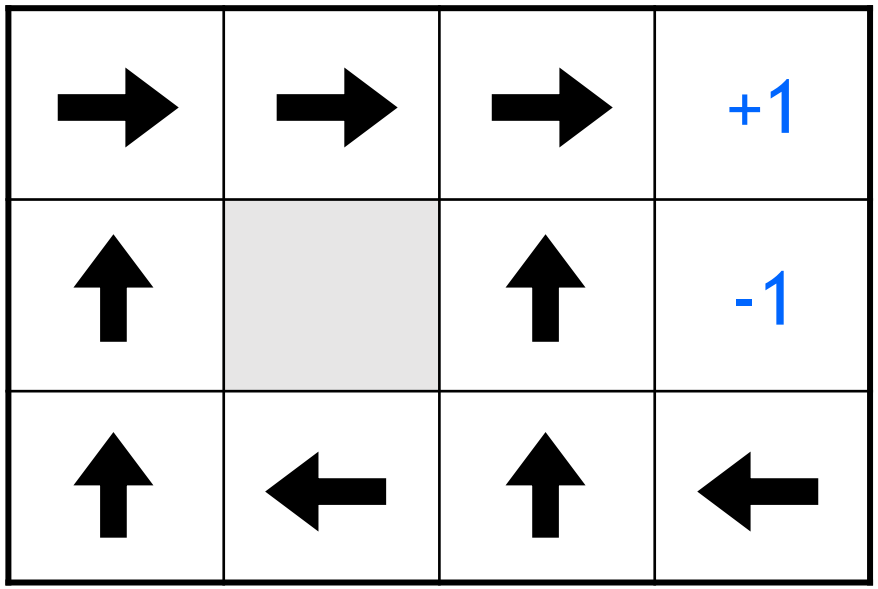


recompensa +1 la [4,3], -1 la [4,2]
recompensa -0.04 altfel

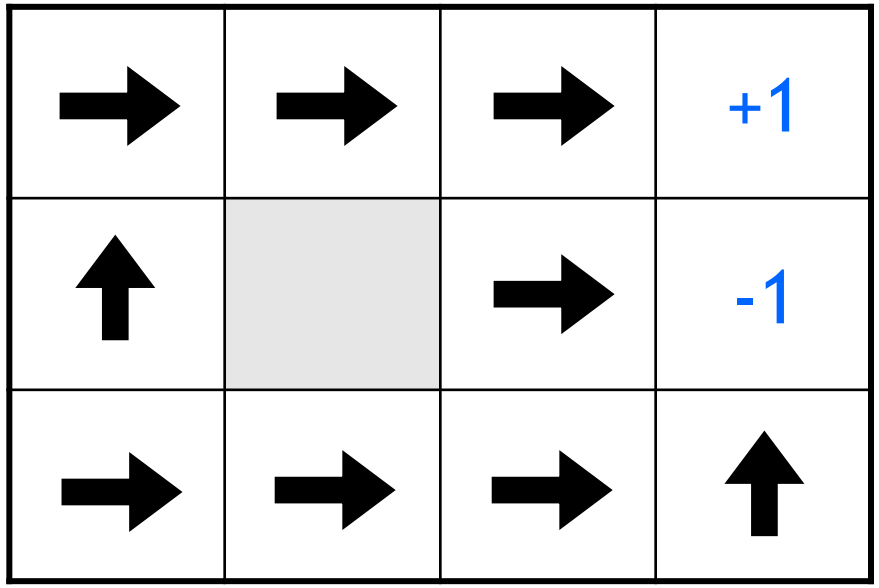
- stari
- actiuni
- recompense

Politica optima

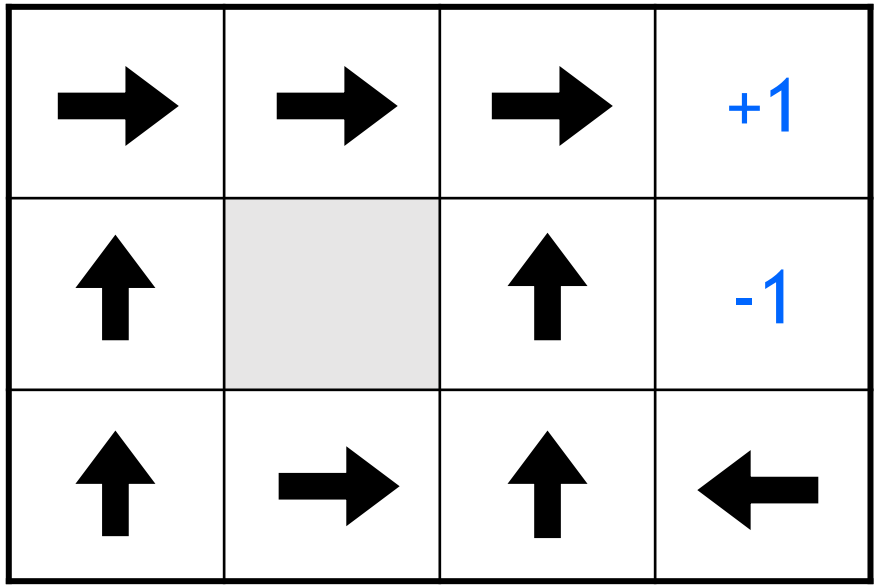




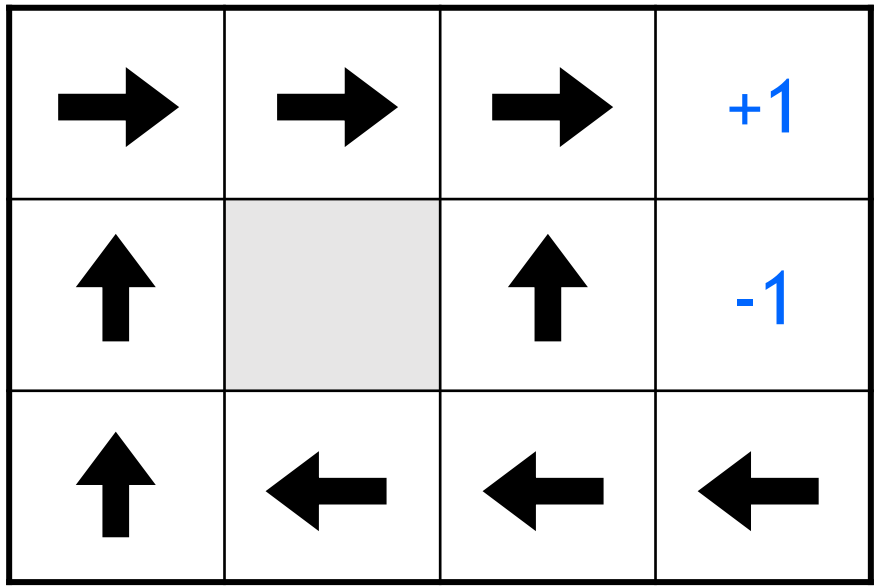
Recompensa pentru fiecare pas: -2



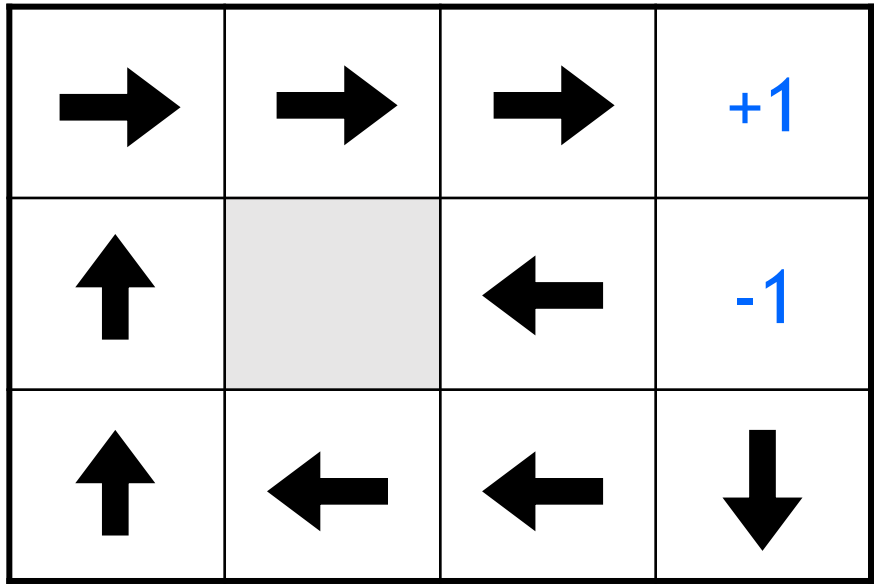
Recompensa pentru fiecare pas: -0.1



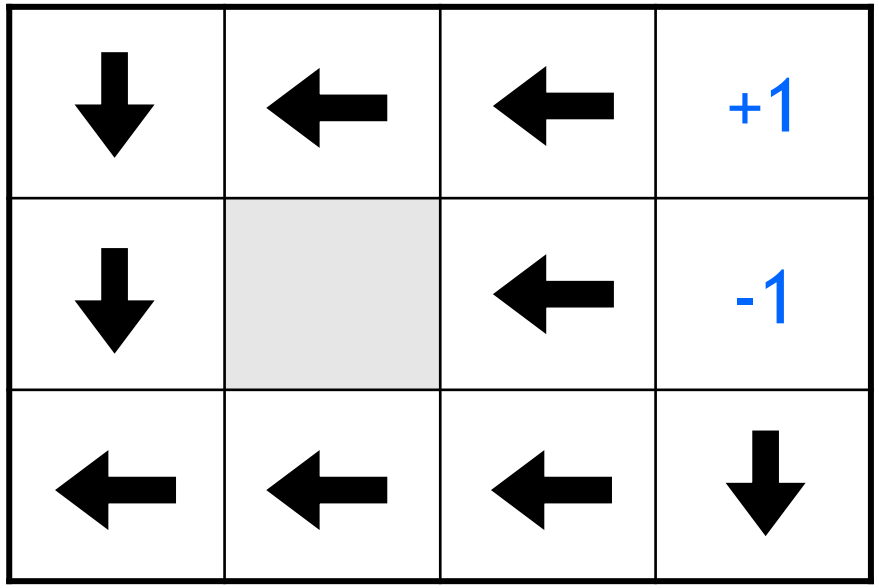
Recompensa pentru fiecare pas : -0.04



Recompensa pentru fiecare pas : -0.01



Recompensa pentru fiecare pas : +0.01



Comportament agent RL

$R(S_t)$ – recompensa pe termen scurt / imediata pentru starea S_t

U – recompensa estimata

- **Orizont finit**: $U(S_t) = (\Sigma_{t=0,\, n}\, R(S_t))$

- **Orizont infinit:**

$$U(S_t) = (\Sigma_{t=0,\infty}\, R(S_t))$$

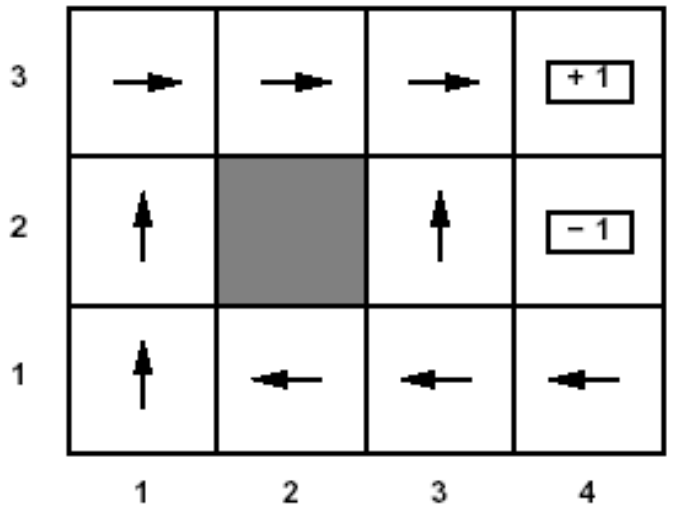
- **Orizont infinit atenuat (gama = δ)**

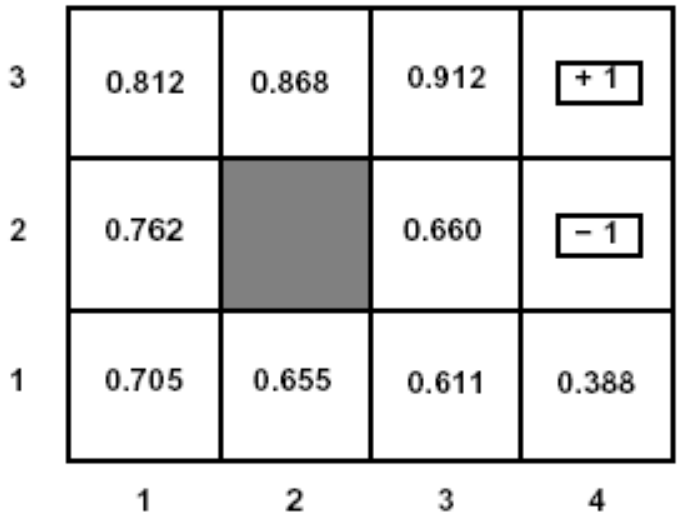
$$U(S_t) = (\Sigma_{t=0,\infty}\, \gamma^t\, R(S_t))$$

U(s) - R(s)

- R(s) recompensa pe termen scurt asociata starii s
- U(s) recompensa pe termen lung din starea s folosind politica **π***

➢ Exemplu: U(s) pentru poliica optima *R(s non-terminal)* = -0.04 si γ =1



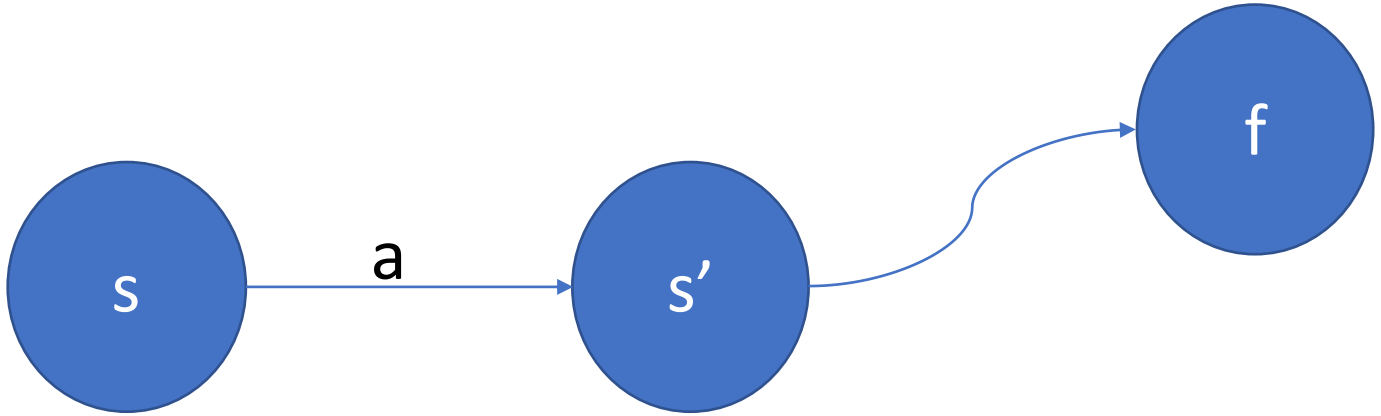


Politica optima

- Se calculeaza $U(S_i)$ pentru fiecare stare S_i

$$U(s) = R(s) + \gamma \max_a [\Sigma_{s'} T(s,a,s') U(s')]$$

- Se opreste daca $|U^t(s)-U^{t+1}(s)| < eps$



Politica optima

- Actiunea cea mai buna in starea S_i este data de

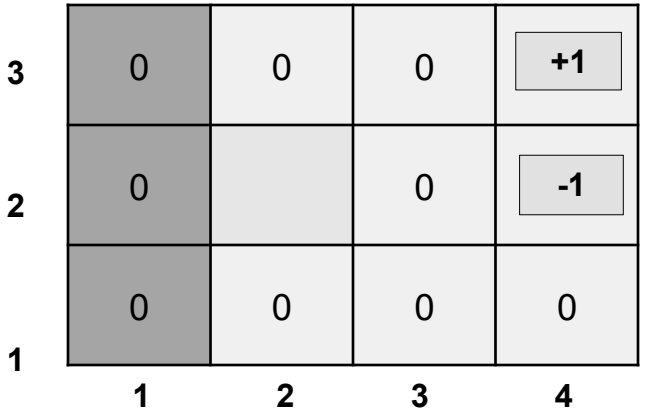
$$\arg\max_a \left(R(S_i) + \gamma \sum_{j=1,n} T(S_i, a, S_j)\, U(S_j)\right)$$

Politica optima: $\pi^*(s) = \arg\max_a \left(\gamma \sum_{j=1,n} T(S_i, a, S_j)\, U(S_j)\right)$

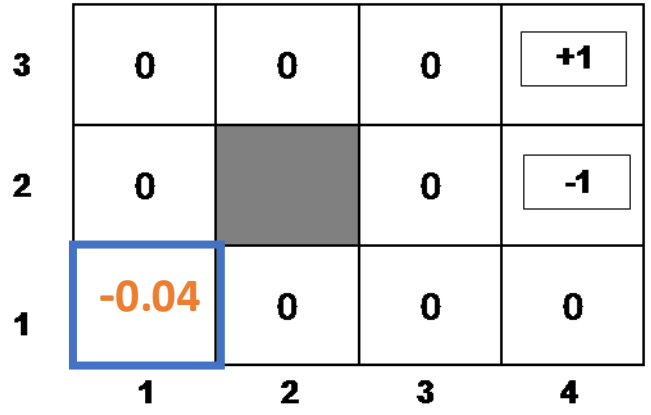
Exemplu

- Daca $U^{(0)}(s) = 0$

Iteration 0



Iteration 1

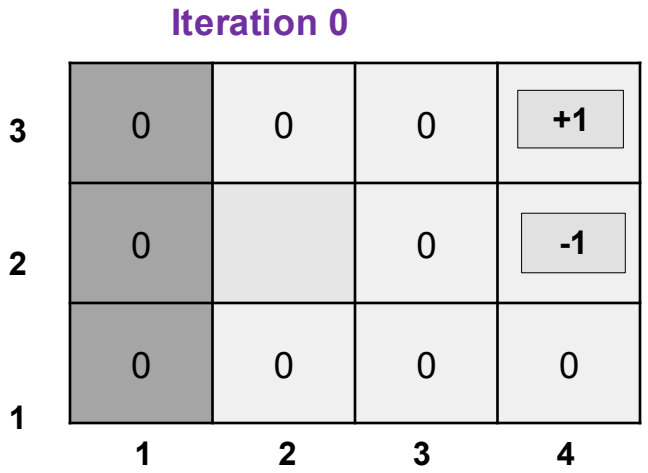


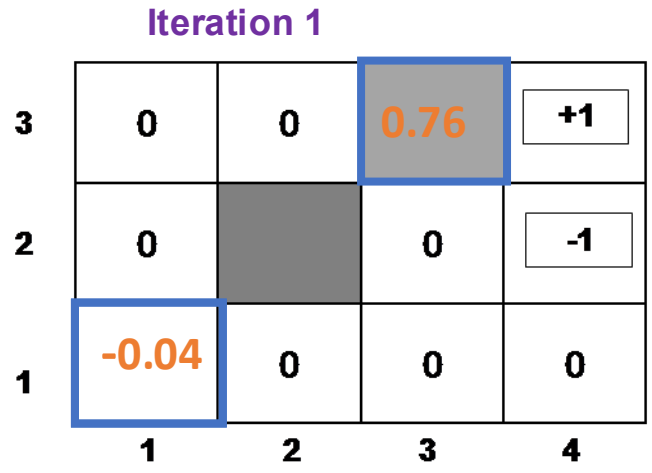
$$U^{(1)}(1,1) = -0.04 + 1 * \max \begin{bmatrix} 0.8U^{(0)}(1,2) + 0.1U^{(0)}(2,1) + 0.1U^{(0)}(1,1) & UP \\ 0.9U^{(0)}(1,1) + 0.1U^{(0)}(1,2) & LEFT \\ 0.9U^{(0)}(1,1) + 0.1U^{(0)}(2,1) & DOWN \\ 0.8U^{(0)}(2,1) + 0.1U^{(0)}(1,2) + 0.1U^{(0)}(1,1) & RIGHT \end{bmatrix}$$

$$U^{(1)}(1,1) = -0.04 + \max \begin{bmatrix} 0 & UP \\ 0 & LEFT \\ 0 & DOWN \\ 0 & RIGHT \end{bmatrix}$$

Exemplu

- $U^{(1)}(3,3)$





$$U^{(1)}(3,3) = -0.04 + 1 * \max \begin{bmatrix} 0.8U^{(0)}(3,3) + 0.1U^{(0)}(2,3) + 0.1U^{(0)}(4,3) & UP \\ 0.8U^{(0)}(2,3) + 0.1U^{(0)}(3,3) + 0.1U^{(0)}(3,2) & LEFT \\ 0.8U^{(0)}(3,2) + 0.1U^{(0)}(2,3) + 0.1U^{(0)}(4,3) & DOWN \\ 0.8U^{(0)}(4,3) + 0.1U^{(0)}(3,3) + 0.1U^{(0)}(3,2) & RIGHT \end{bmatrix}$$

$$U^{(1)}(3,3) = -0.04 + \max \begin{bmatrix} 0.1 & UP \\ 0 & LEFT \\ 0.1 & DOWN \\ 0.8 & RIGHT \end{bmatrix}$$

Exemplu

- $U^{(1)}(4,1)$

Iteratia 0

3	0	0	0	+1
2	0		0	-1
1	0	0	0	0
	1	2	3	4

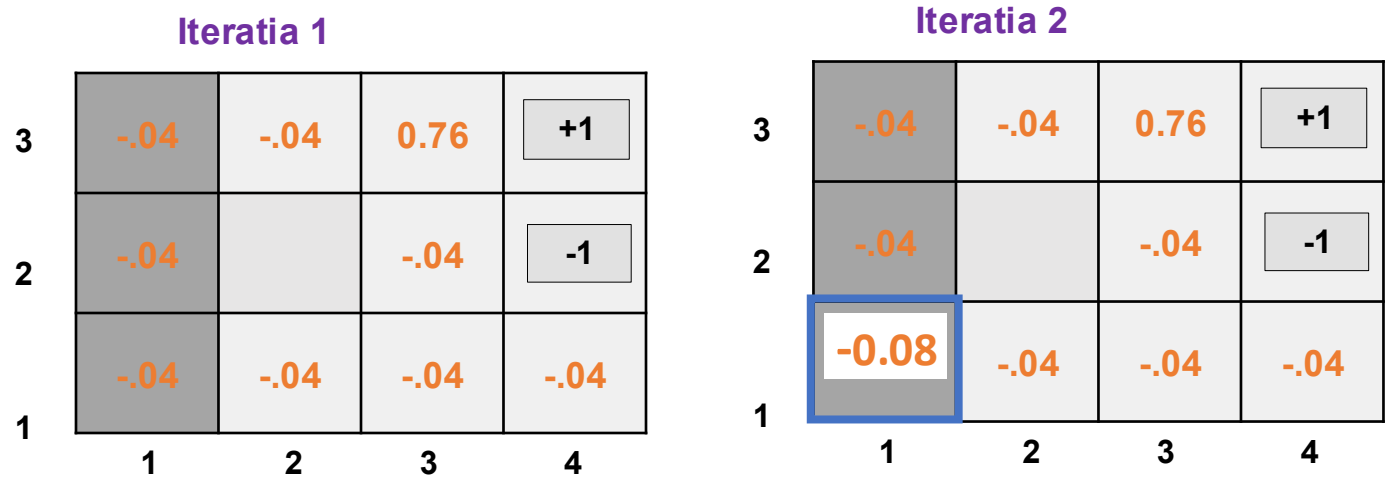
Iteratia 1

3	0	0	.76	+1
2	0		0	-1
1	-0.04	0	0	-0.04
	1	2	3	4

$$U^{(1)}(4,1) = -0.04 + \max \begin{bmatrix} 0.8U^{(0)}(4,2) + 0.1U^{(0)}(3,1) + 0.1U^{(0)}(4,1) & UP \\ 0.8U^{(0)}(3,1) + 0.1U^{(0)}(4,2) + 0.1U^{(0)}(4,1) & LEFT \\ 0.8U^{(0)}(4,1) + 0.1U^{(0)}(3,2) + 0.1U^{(0)}(4,1) & DOWN \\ 0.8U^{(0)}(4,1) + 0.1U^{(0)}(4,2) + 0.1U^{(0)}(4,1) & RIGHT \end{bmatrix}$$

$$U^{(1)}(4,1) = -0.04 + \max \begin{bmatrix} -0.8 & UP \\ -0.1 & LEFT \\ 0 & DOWN \\ -0.1 & RIGHT \end{bmatrix}$$

Pasi in iteratia a doua

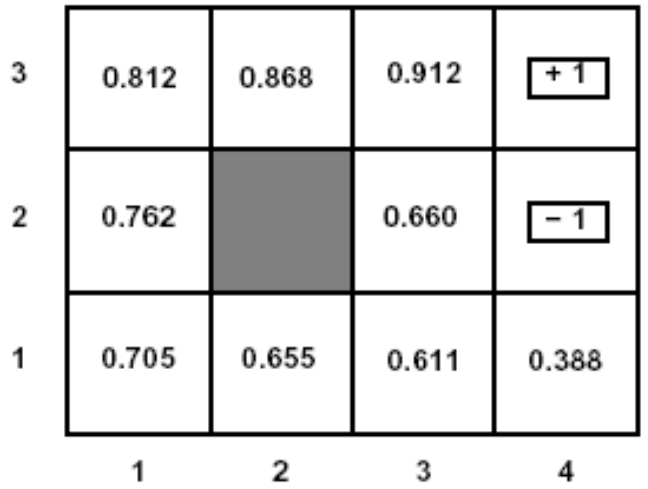


$$U^{(2)}(1,1) = -0.04 + 1 * \max \begin{bmatrix} 0.8U^{(1)}(1,2) + 0.1U^{(1)}(2,1) + 0.1U^{(1)}(1,1) & UP \\ 0.9U^{(1)}(1,1) + 0.1U^{(1)}(1,2) & LEFT \\ 0.9U^{(1)}(1,1) + 0.1U^{(1)}(2,1) & DOWN \\ 0.8U^{(1)}(2,1) + 0.1U^{(1)}(1,2) + 0.1U^{(1)}(1,1) & RIGHT \end{bmatrix}$$

$$U^{(2)}(1,1) = -0.04 + \max \begin{bmatrix} -.04 & UP \\ -.04 & LEFT \\ -.04 & DOWN \\ -.04 & RIGHT \end{bmatrix} = -0.08$$

Exemplu

$$\pi^*(s) = \arg\max_a \sum_{s'} P(s' \mid s, a) U(s')$$

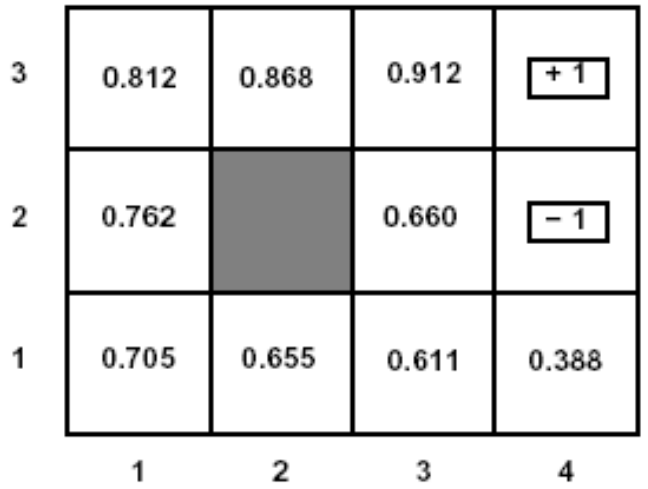


- Actiunea cea mai buna in (1,1)

$$\pi^*(1,1) = \arg\max \begin{bmatrix} 0.8U(1,2) + 0.1U(2,1) + 0.1U(1,1) & UP \\ 0.9U(1,1) + 0.1U(1,2) & LEFT \\ 0.9U(1,1) + 0.1U(2,1) & DOWN \\ 0.8U(2,1) + 0.1U(1,2) + 0.1U(1,1) & RIGHT \end{bmatrix}$$

Exemplu

$$\pi^*(s) = \arg\max_a \sum_{s'} P(s' \mid s, a) U(s')$$



- Actiunea cea mai buna in(1,1)

$$\pi^*(1,1) = \arg\max \begin{bmatrix} 0.8U(1,2) + 0.1U(2,1) + 0.1U(1,1) & UP \\ 0.9U(1,1) + 0.1U(1,2) & LEFT \\ 0.9U(1,1) + 0.1U(2,1) & DOWN \\ 0.8U(2,1) + 0.1U(1,2) + 0.1U(1,1) & RIGHT \end{bmatrix}$$

Value iteration

1. Initializeaza $U^1(S_i)$ pentru fiecare S_i, $k \leftarrow 1$ ($U^1(S_i)=0$)

2. repeat

$k \leftarrow k + 1$

Calculeaza $U^k(S_i)$ for each S_i $\quad U^k(s) = R(s) + \gamma \max_a [\sum_{s'} T(s,a,s') U^{k-1}(s')]$

until $\max_i |U^k(S_i) - U^{k-1}(S_i)| < eps$

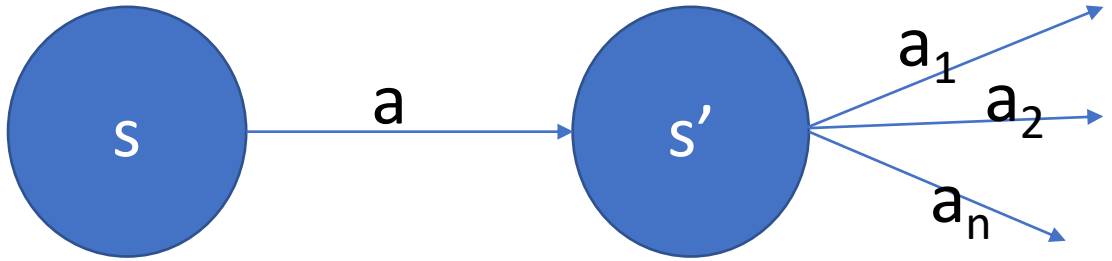
Q-learning

- Explorarea mediului in vederea invatarii utilitatilor
- Se invata actiuni – valori
- Q: S x A $\rightarrow$ U
- **Q(s, a)** – estimarea sumei pentru recompensele viitoare pornind din starea **s** si alegerea actiunii **a** pentru politica optima
- Q este corelata cu utilitatile

$$U(s) = \max_a Q(s, a)$$

Q-learning

$$Q(s, a) \leftarrow Q(s, a) + \alpha(R(s) + \delta \max_{a'} Q(s', a') - Q(s, a))$$



Se calculeaza pentru fiecare tranzitie din starea s in starea s'

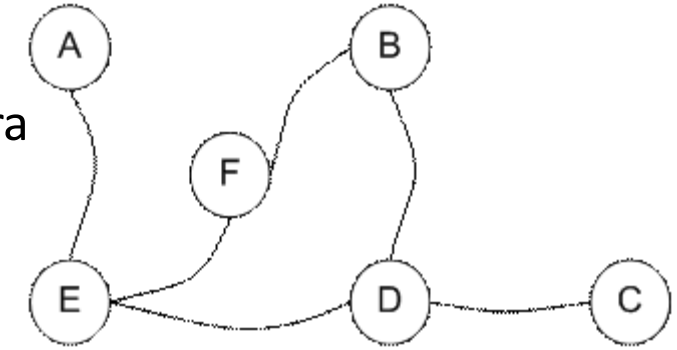
$$Q(s, a) \leftarrow Q(s, a)(1 - \alpha) + \alpha(R(s) + \delta \max_{a'} Q(s', a'))$$

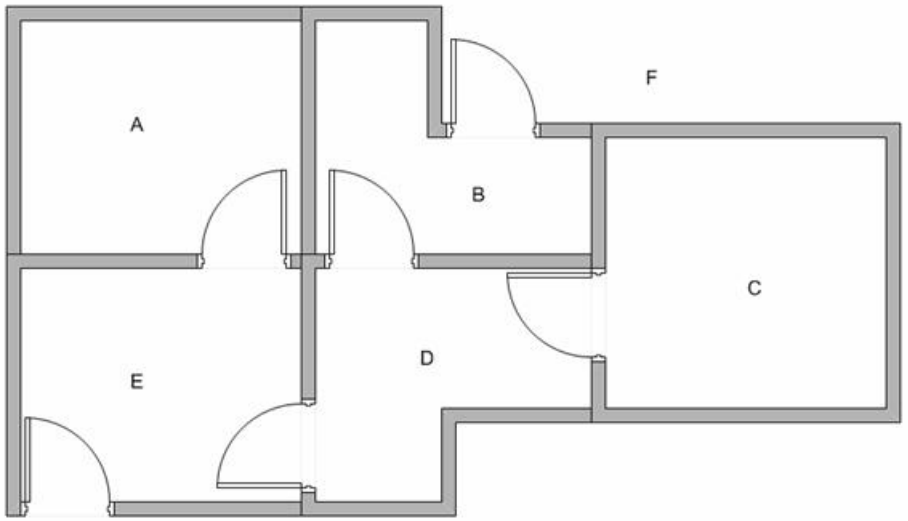
δ - factor atenuare

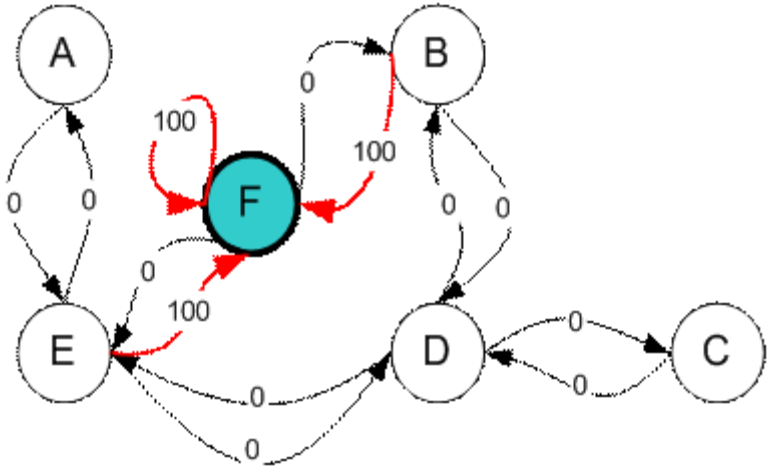
α – learning rate

Exemplu Q-learning (α=1)

- 5 camere: A .. E, F (exterior)
- Camere – noduri in graf, usi – laturile grafului
- Scop – sa se ajunga in exterior plecand din fiecare camera
- F->exterior – recompensa 100, altfel 0
- Usa are 2 directii => 2 laturi



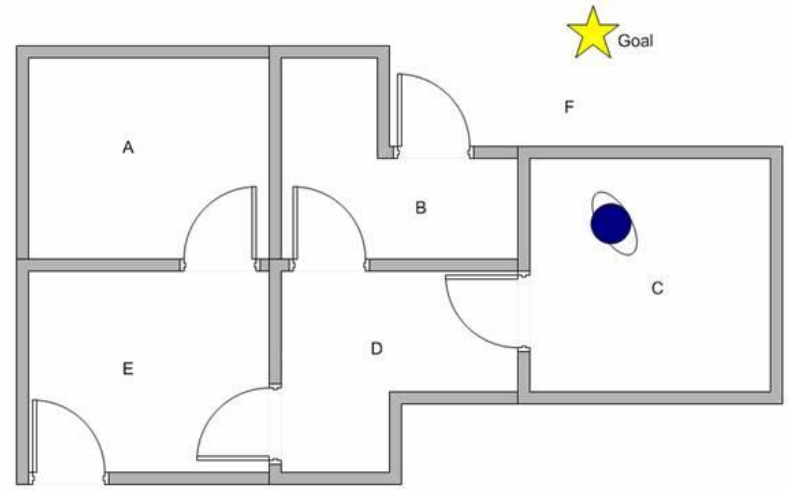


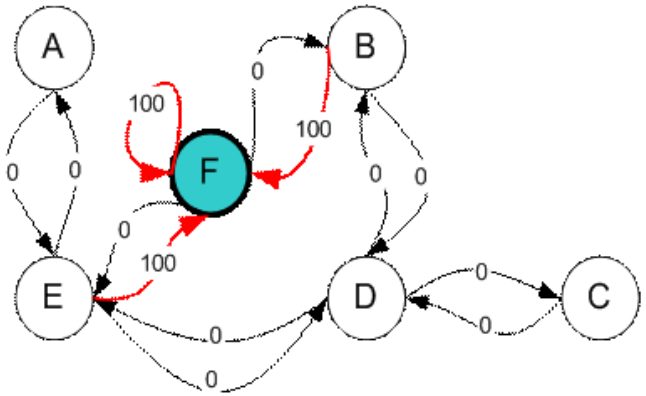


F – scope state

http://people.revoledu.com/kardi/tutorial/ReinforcementLearning/index.html

C -> F?





$$
\begin{array}{c} \\ A \\ B \\ C \\ D \\ E \\ F \end{array}
\begin{array}{c}
\begin{array}{cccccc} A & B & C & D & E & F \end{array} \\
\left[\begin{array}{cccccc}
- & - & - & - & 0 & - \\
- & - & - & 0 & - & 100 \\
- & - & - & 0 & - & - \\
- & 0 & 0 & - & 0 & - \\
0 & - & - & 0 & - & 100 \\
- & 0 & - & - & 0 & 100
\end{array}\right]
\end{array}
= R
$$

Q-learning

1. Initializare δ, (α=1) si **R**
2. Initializare **Q** cu 0
3. Pentru fiecare episod
   - selecteaza starea initiala (aleator) - s
   - **while** **s** nu e stare de scop **repeta**
      - obtine recompensa **R(s)**
      - selecteaza o actiune **a** din starea **s**
      - executa actiunea **a** din starea s in **s'**
      - $\max_{a'} Q(s',a')$ = valoarea maxima in s' pentru fiecare a'
      - actualizeaza $Q(s,a) \leftarrow R(s) + \delta \max_{a'} Q(s',a')$
      - $s \leftarrow s'$

$$Q(s, a) \leftarrow Q(s, a)(1 - \alpha) + \alpha(R(s) + \delta \max_{a'} Q(s', a'))$$

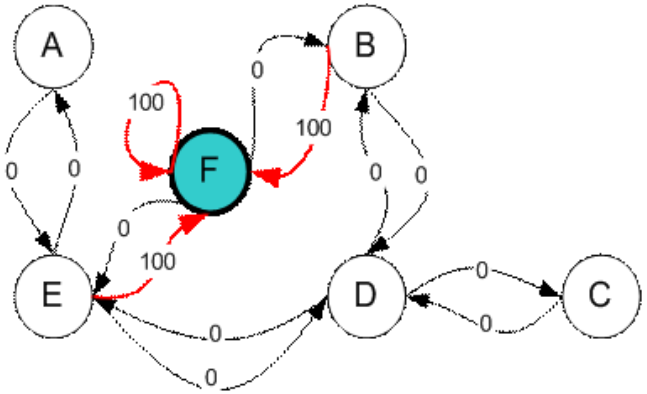
$$\alpha = 1$$

$$Q(s, a) \leftarrow R(s) + \delta \max_{a'} Q(s', a')$$

C -> F

Utilizare Q

1. s ← stare initiala
2. Determina actiunea **a** pentru Q(s,a) este max
3. Executa **a**
4. **Repeta** 2 **pana** la starea de scop



$\delta = 0.8$

Q-learning

1. Initializare δ, (α=1) si **R**
2. Initializare **Q** cu 0
3. Pentru fiecare episod
   - selecteaza starea initiala (aleator) - s
   - **while** **s** nu e stare de scop **repeta**
      - obtine recompense **R(s)**
      - **selecteaza** o actiune **a** din starea **s**
      - executa actiunea **a** din starea s in **s'**
      - $\max_{a'} Q(s',a')$ = valoarea maxima in s' pentru fiecare a'
      - actualizeaza $Q(s,a) \leftarrow R(s) + \delta \max_{a'} Q(s',a')$
      - $s \leftarrow s'$

$$Q(s, a) \leftarrow Q(s, a)(1 - \alpha) + \alpha(R(s) + \delta \max_{a'} Q(s', a'))$$

$$\alpha = 1$$

$$Q(s, a) \leftarrow R(s) + \delta \max_{a'} Q(s', a')$$

Selecteaza a:

- Random
- Selecteaza a – Q(s,a) max -> exploatare
- Eps-greedy : eps -> combinare explorare si exploatare

Q-learning

1. Initializare δ, (α=1) si **R**
2. Initializare **Q** cu 0
3. Pentru fiecare episod
   - selecteaza starea initiala (aleator) - s
   - **while** s nu e stare de scop **repeta**
      - obtine recompense **R(s)**
      - **selecteaza** o actiune **a** din starea **s**
      - executa actiunea **a** din starea s in **s'**
      - $\max_{a'} Q(s',a')$ = valoarea maxima in s' pentru fiecare a'
      - actualizeaza $Q(s,a) \leftarrow R(s) + \delta \max_{a'} Q(s',a')$
      - $s \leftarrow s'$

$$Q(s, a) \leftarrow Q(s, a)(1 - \alpha) + \alpha(R(s) + \delta \max_{a'} Q(s', a'))$$

$$\alpha = 1$$

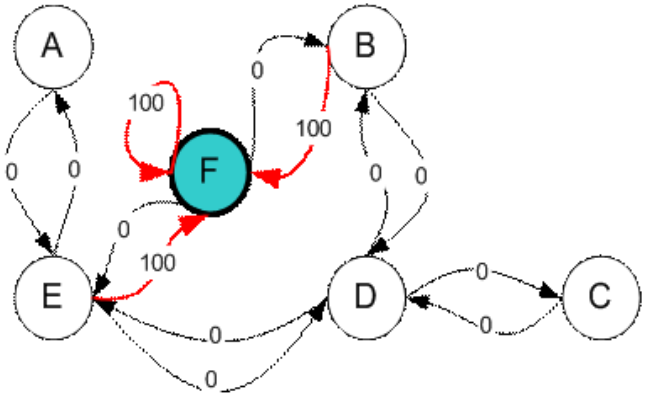
$$Q(s, a) \leftarrow R(s) + \delta \max_{a'} Q(s', a')$$

```
n ← uniform random number between 0 and 1;
if n < ε then
    A ← random action from the action space;
else
    A ← maxQ(S,.);
end
```

Episod – agent in B

Actiuni B -> D and B -> F

Alege aleator B -> F, recompensa 100



$$Q(B,F) = R(B, F) + 0.8 * \max(Q(F,B), Q(F,E), Q(F,F)) = 100 + 0.8 * 0 = 100$$

	A	B	C	D	E	F
A	–	–	–	–	0	–
B	–	–	–	0	–	100
C	–	–	–	0	–	–
D	–	0	0	–	0	–
E	0	–	–	0	–	100
F	–	0	–	–	0	100

= R

	A	B	C	D	E	F
A	0	0	0	0	0	0
B	0	0	0	0	0	100
C	0	0	0	0	0	0
D	0	0	0	0	0	0
E	0	0	0	0	0	0
F	0	0	0	0	0	0

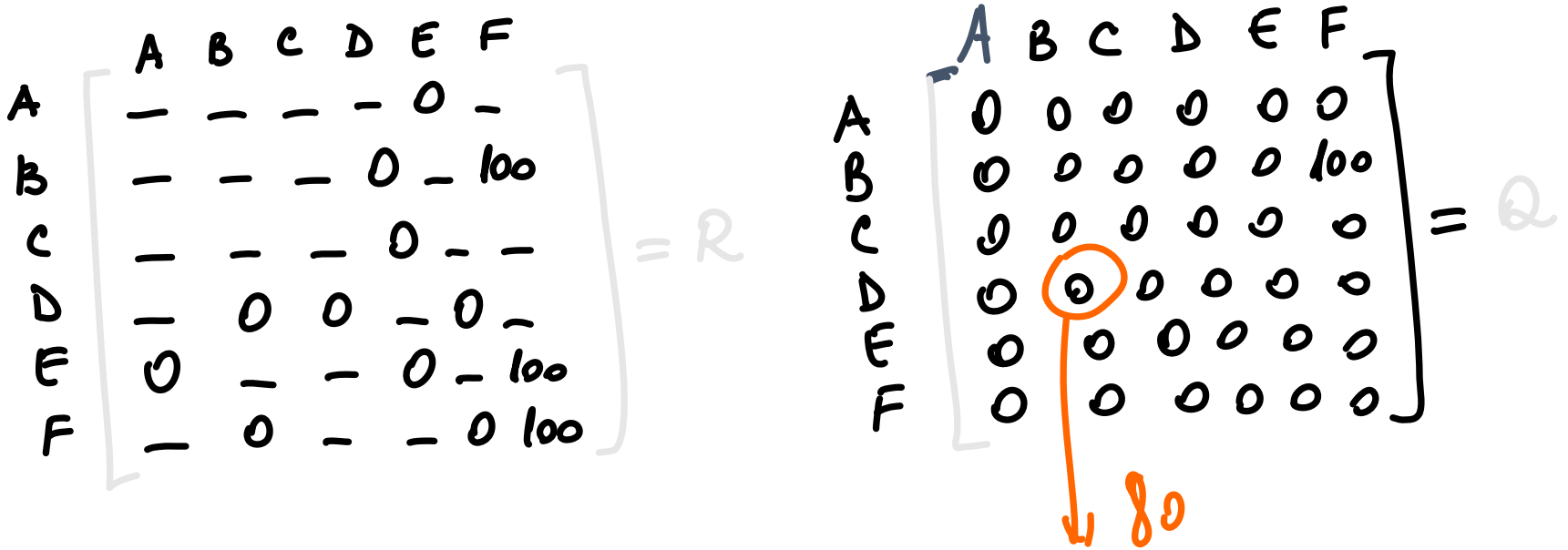
= Q

Episod – agent in D

actiuni D->B, D->C and D->E

Alege aleator D ->B, recompensa 0

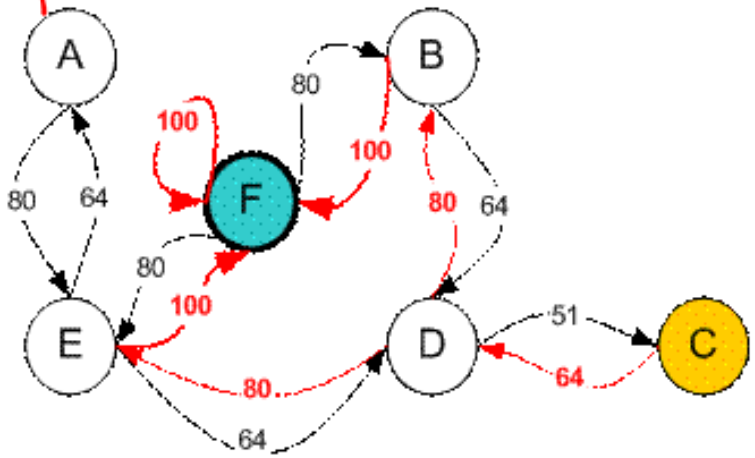
$$Q(D, B) = R(D, B) + 0.8 * \max(Q(B,D), Q(B,F)) = 0 + 0.8 * \max(0, 100) = 80$$



Convergenta

Q

stare/act	A	B	C	D	E	F
A	–	–	–	–	400	0
B	–	–	–	320	–	500
C	–	–	–	320	–	–
D	–	400	256	–	400	–
E	320	–	–	320	–	500
F	–	400	–	–	400	500



Normalizare

Q

stare/act	A	B	C	D	E	F
A	–	–	–	–	80	–
B	–	–	–	64	–	100
C	–	–	–	64	–	–
D	–	80	51	–	80	–
E	64	–	–	64	–	100
F	–	80	–	–	80	100

C –> D

D –> B sau E, alege aleator B

B -> D sau F, alege F

C -> D -> B -> F

Evaluare Q-learning

- Asigurare convergenta in vederea obtinerii unei politici optime.
- Evaluare eficienta si stabilitate.
- **Fine-tuning pentru parametri:**
  - viteza de invatare,
  - factor de atenuare, etc
- Identificare limitari si posibile imbunatatiri.

Evaluare Q-learning

1. **Viteza de convergenta** – cat de repede se stabilizeaza valorile din Q.
2. **Cumulative Reward** – se masoara recompensa totala dupa episoadele realizate.
3. **Stabilitate Q** – se verifica daca valorile din Q la convergenta fluctueaza.
4. **Acuratetea politicii optime** – Compararea ceea ce s-a invatat versus cea mai buna politica teoretica.
5. **Explorare vs. Exploatare** – se cauta o balanta optima.

Masurare viteza de convergenta

- Analiza valorilor din Q in timp.
- Daca modificarea valorilor nu este semnificativa -> modelul a ajuns la convergenta.
- Folosirea MSD - **Mean Squared Difference** aplicata pe valorile din Q pentru iteratii consecutive.

$$MSD = \frac{1}{N} \sum (Q_{t+1} - Q_t)^2$$

- MSD ~ 0 -> s-a ajuns la convergenta

Masurare viteza de convergenta

- Analiza valorilor din Q in timp:
- Daca valorile fluctueaza: se modifica viteza de invatare.
- Se verifica **overfitting** (memorizeaza pe datele pe care invata in loc sa generalizeze)
- Validare comparativ la valorile reale (daca exista).

Masurare viteza de convergenta

Analiza recompense cumulate de-a lungul episoadelor

- Trasare recompense totala / episod pentru a vizualiza trendul
- Daca graficul recompense cumulate se stabileaza / ajunge la platou -> s-a ajuns la convergenta
- Fluctuatii -> instabilitate

Masurare viteza de convergenta

Analiza stabilitatii politicii

- Compararea actiunilor selectate in diferite episoade
- Daca politica (distributia actiunilor) nu se modifica semnificativ -> s-a ajuns la convergenta

Evaluarea performantelor intr-un mediu de test

- Rulare model invatat pe un alt mediu de test
- Se verifica stabilitatea politicii
- Daca performantele variaza semnificativ -> necesar mai multe episode de antrenare

Evaluare recompensa cumulata

- Recompense mai mari -> politici mai bune
- Se analizeaza recompensa totala / episode pentru a se observa trendul
- Se ajunge la un platou de valori -> s-a ajuns la convergenta

- Comparare performante pentru diferiti parametri:
  - Viteza de invatare mare: poate preveni convergenta (fluctuarea valorilor din Q)
  - Viteza de invatare mica: poate incetini convergenta

Analiza explorare vs. exploatare

- Asigurare explorare suficienta – se incearca utilizarea mai devreme a actiunilor noi
- Reducerea caracterului aleator **(ε din startegia ε-greedy)**
- Compararea rezultatelor folosind diferite strategii (random, max, etc).

Imbunatatire Q-Learning

- Rezolvarea de sarcini complexe
- Deep Q-learning - inlocuieste Q cu o retea neurala pentru a estima valorile Q